

Distributed Cache

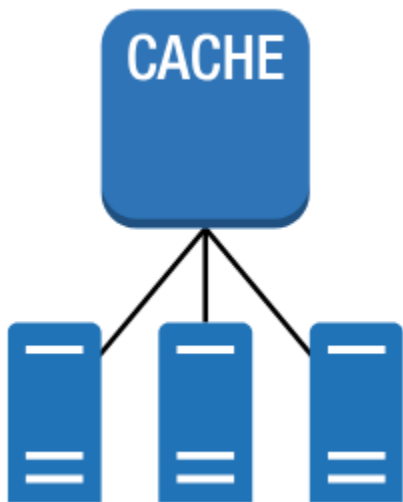
1. Overview

A **Distributed Cache** is a shared high-speed cache cluster used by multiple application nodes to store and retrieve frequently accessed data.

Common examples:

- Redis
- Memcached

Instead of each service instance storing data locally, all nodes use a centralized/shared cache layer.



2. Why Use Distributed Cache

Without cache:

None

App Server → Database / Origin Service

Repeatedly hitting database increases:

- Latency
- Cost
- Load
- Scalability pressure

With distributed cache:

None

App Server → Redis Cache → Fast Response

Only cache miss goes to origin.

3. In Context of Broadcasting / Shared Data

Suppose multiple service nodes need common updated data.

Instead of full mesh communication:

None

Node A sends to Node B,C,D directly

Use distributed cache:

None

Node A writes latest state to Redis

Node B reads Redis

Node C reads Redis

Node D reads Redis

Shared state propagation becomes simpler.

4. Core Architecture

None

Origin DB / Source System



Service Nodes periodically fetch updates



Write updates to Distributed Cache



Other nodes read latest data

5. Why It Helps Performance

Distributed cache is usually:

- In-memory
- Network-accessible
- Extremely fast
- Shared across instances

Typical latency:

None

Microseconds to low milliseconds

Much faster than many databases.

6. Example Use Cases

A. User Sessions

Store login session centrally.

B. Product Catalog

Frequently viewed product data.

C. Feature Flags

All nodes read latest config.

D. Counters

Views, likes, rate limits.

E. Shared Metadata

Leaderboard, rankings.

7. Example: Multiple App Servers

None

App1

App2

App3



Redis Cluster

If App2 scales to 20 instances, same cache still usable.

8. Independent Scaling Advantage

App layer and cache layer can scale separately.

None

Traffic grows:

Scale app servers

Cache hot keys increase:

Scale Redis cluster

This is strong HLD design flexibility.

9. Compared to Full Mesh

Full Mesh

Each node communicates with every node.

Distributed Cache

Each node communicates only with cache.

Message complexity lower than mesh.

10. Polling vs Push Model

Polling

Nodes periodically fetch new data.

None

Every 5 sec read cache

Simple but delayed.

Push / PubSub

Use Redis pub/sub or streams.

None

Update event pushed instantly

Lower delay.

11. Cache Consistency Challenge

Caches may become stale.

Example:

None

DB updated

Cache not updated yet

Need strategies:

- TTL expiry
- Write-through
- Write-behind
- Cache invalidation
- Pub/sub updates

12. More Requests Trade-off

Compared to direct host-host messages:

Using cache may add extra operations:

None

Write to cache

Then reads from many nodes

But often still worth it for simplicity and speed.

13. Validation / Schema Risk (Important)

When services call APIs directly:

Receiver can validate request body and reject invalid payload.

Example:

None

HTTP 400 / 422

Immediate detection.

But cache is just storage.

A node may write:

None

Malformed JSON

Missing fields

Wrong types

Unexpected structure

Other nodes discover later when reading.

This delays detection.

14. Security Risks

Caches are often designed for trusted internal networks.

Risks:

- Open ports exposed publicly
- Unauthorized reads/writes
- Data poisoning
- Remote command misuse
- Memory dumps exposing data

So production cache should be:

- In private subnet
- Authenticated
- Network restricted
- Monitored

15. Encryption Considerations

Some cache systems historically had limited built-in encryption support depending on deployment mode.

Need to consider:

In Transit

TLS between app and cache.

At Rest

Disk persistence encryption if enabled.

Trade-offs:

- More CPU cost
- Slight latency overhead
- Operational complexity

16. Cache Data Structures

Redis supports:

- Strings
- Hashes
- Lists
- Sets
- Sorted sets
- Streams
- Bitmaps

Useful beyond plain cache.

Memcached is simpler key-value memory cache.

17. Failure Scenarios

Cache Node Down

Need replication / cluster failover.

Cache Miss Storm

After restart, many requests hit DB (thundering herd).

Hot Key

One popular key overloads node.

Need sharding / replication.

18. Cache Patterns

A. Cache Aside

App checks cache first.

B. Write Through

Write DB + cache together.

C. Write Back

Write cache first, DB later.

D. Read Through

Cache loads data automatically.

19. Distributed Cache vs Coordination Service

Feature	Distributed Cache	Coordination Service
Speed	Very fast	Moderate
Simplicity	Easier	Harder
Strong consistency	Weak/limited	Stronger
Locks / leader election	Basic/custom	Native
Metadata safety	Medium	High

20. Interview Answer

Q: Why use Redis in system design?

Answer:

I'd use Redis as a distributed cache for low-latency shared data like sessions, counters, rate limits, and hot reads. It reduces DB load and scales independently. I'd also discuss cache invalidation, replication, failover, and security hardening.

21. Best Practices

- Keep cache in private network
- Enable auth/TLS where available
- Set TTLs
- Avoid storing sensitive data casually
- Use replication
- Monitor hit ratio
- Handle cache misses gracefully
- Validate serialized payload schemas

22. Short Revision Notes

None

Distributed Cache = Shared in-memory cache cluster

Examples:

Redis, Memcached

Uses:

- Sessions
- Counters
- Product cache
- Shared metadata

Pros:

Fast, scalable, low latency

Cons:

Stale data, security, invalidation